SECURITY MONITORING / HANDS-ON CASE STUDY

Runes Linux Detection Lab

Authentication monitoring, brute-force investigation and log correlation

A practical SOC-style investigation using Apache HTTP access logs and application-level authentication events to identify repeated credential-guessing activity and verify successful logins during the test window.

ENVIRONMENT **Kali Linux • local controlled lab**	APPLICATION STACK **Apache2 • PHP • MariaDB**
TESTING TOOL **Burp Suite Community • Intruder**	PRIMARY TELEMETRY **access.log • runes_auth.log**

KEY OUTCOME
The captured evidence shows repeated POST requests to /login.php, numerous failed authentication events and six successful authentication events visible in the filtered log excerpts. Those successes align with corresponding HTTP 302 responses in Apache access.log.

Publication note: session tokens and a password value are redacted in request screenshots. All testing was performed in a controlled lab against the author's test application.

01 / INVESTIGATION OVERVIEW

Executive Summary

This project demonstrates a small, repeatable security-monitoring workflow for a PHP/MariaDB login application hosted by Apache2. A normal-activity baseline was captured first. A controlled Burp Suite Intruder Cluster bomb run then generated combinations of usernames and passwords, producing a dense series of login requests and both failed and successful authentication events.

Scope and environment

Item	Description
Objective	Observe, detect and investigate automated credential-guessing activity in a controlled web-application lab.
In scope	Login endpoint, normal authentication baseline, failed-login burst, Intruder results, application-authentication log and Apache access log.
Tools	Kali Linux, Apache2, PHP, MariaDB, Burp Suite Community Edition, grep, awk, sort and uniq.
Primary evidence	/var/log/apache2/access.log and /var/log/apache2/runes_auth.log.

Investigation workflow

1. Establish expected service and login behavior before generating test traffic.
2. Use Intruder to submit a controlled set of username/password combinations.
3. Review access.log for repeated POST /login.php requests, status codes and response-size patterns.
4. Review runes_auth.log for explicit LOGIN_FAILURE and LOGIN_SUCCESS events, usernames, timestamps and source IPs.

02 / ENVIRONMENT

Lab Setup and Application Baseline

The test environment uses Apache2 to serve the Runes login page, PHP to process submitted credentials and MariaDB to retrieve user records. The application uses a prepared statement to look up a username and password_verify() to check the submitted password against the stored password hash.

01 CLIENT	02 WEB	03 DATA	04 TELEMETRY
Browser / Burp Suite	Apache2 + PHP	MariaDB / users	access.log + auth log

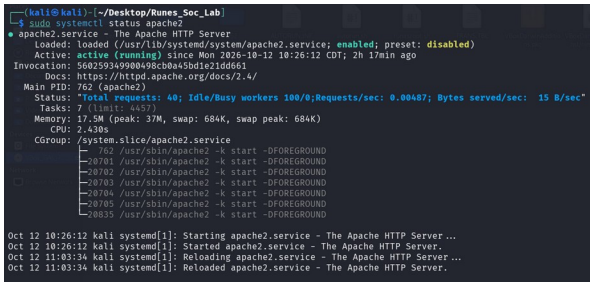


FIGURE 1 | Apache2 service

Service status confirms the web server is running for the test scenario.



FIGURE 2 | MariaDB service

Database service is running so the login handler can query the users table.

Application entry point: login.php

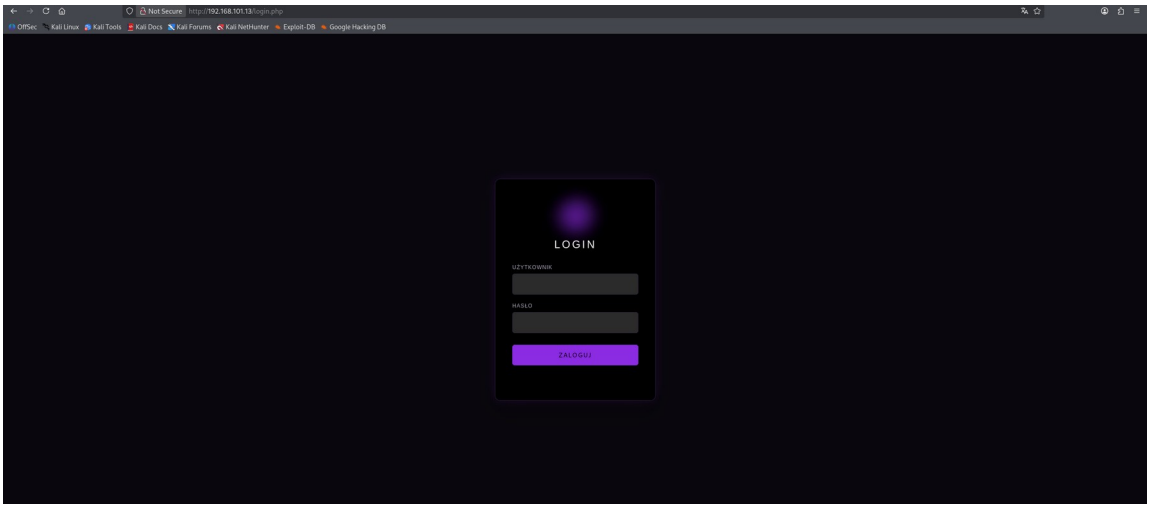


FIGURE 3 | Runes login interface

The browser page exposes username and password fields and submits the form to login.php. The interface itself does not establish authentication success; the application log provides that outcome.

> **BASELINE PRINCIPLE**
>
> Capture normal service behavior and at least one expected successful and failed login before running Intruder. This gives the analyst a comparison point for later anomalies.

03 / NORMAL BEHAVIOR

Baseline Telemetry and Authentication Logging

The first baseline requests show why web-server logs and application authentication logs answer different questions. Apache records the HTTP transaction; the custom JSON log records the result of the credential check.

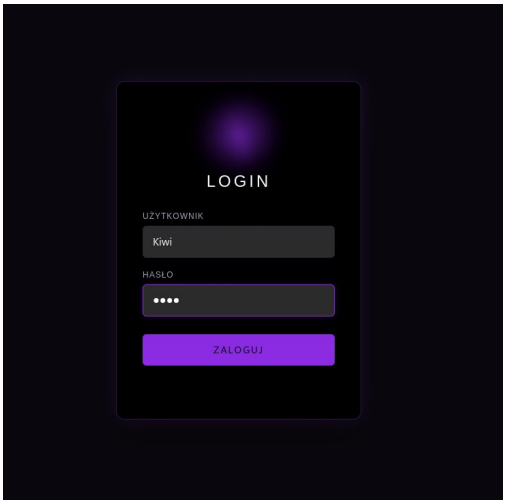


FIGURE 4 | Test login input

A test account is entered in the login form. This screenshot documents the test action only; success is confirmed by the corresponding application log event.

Apache access.log

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ tail -f /var/log/apache2/access.log
192.168.101.13 - - [14/Oct/2026:11:02:37 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:03:07 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64;
192.168.101.13 - - [14/Oct/2026:11:03:08 -0500] "GET /index.html HTTP/1.1" 200 3999 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:03:37 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:03:46 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64;
192.168.101.13 - - [14/Oct/2026:11:03:46 -0500] "GET /index.html HTTP/1.1" 200 3999 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:03:55 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:04:09 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64;
192.168.101.13 - - [14/Oct/2026:11:04:10 -0500] "GET /index.html HTTP/1.1" 200 3999 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:04:20 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:04:32 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
192.168.101.13 - - [14/Oct/2026:11:04:39 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_64
```

FIGURE 5 | Baseline HTTP requests

The access log shows timestamps, client address, HTTP method/path, status code, response size and user-agent data. A 302 response is consistent with the application redirect, but must be correlated with LOGIN_SUCCESS.

Application authentication log

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ tail -f runes_auth.log
{"timestamp":"2026-10-14T16:02:37+00:00","event":"LOGIN_FAILURE","username":"wwd","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:03:08+00:00","event":"LOGIN_SUCCESS","username":"Mixu","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:03:37+00:00","event":"LOGIN_FAILURE","username":"wwd","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:03:46+00:00","event":"LOGIN_SUCCESS","username":"Wielo","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:03:55+00:00","event":"LOGIN_FAILURE","username":"wwd","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:04:10+00:00","event":"LOGIN_SUCCESS","username":"Socik","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:04:20+00:00","event":"LOGIN_FAILURE","username":"wwd","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:04:32+00:00","event":"LOGIN_FAILURE","username":"Daniel","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:04:39+00:00","event":"LOGIN_FAILURE","username":"Daniel","ip":"192.168.101.13","reason":"unknown_user"}
```

FIGURE 6 | Structured authentication events

The application writes JSON events with timestamp, event name, username, source IP and (for failures) a reason such as unknown_user. This is the primary evidence for authentication outcome.

> **DATA SOURCE DISTINCTION**
>
> access.log does not contain the application's authentication decision by default. runes_auth.log does, because the login handler explicitly emits LOGIN_SUCCESS and LOGIN_FAILURE events.

04 / REQUEST INSPECTION

Supplementary Input Probe and Request Capture

A separate input-handling check submitted an XSS-shaped string to the username field. It is documented as a probe only and is not part of the brute-force conclusion.

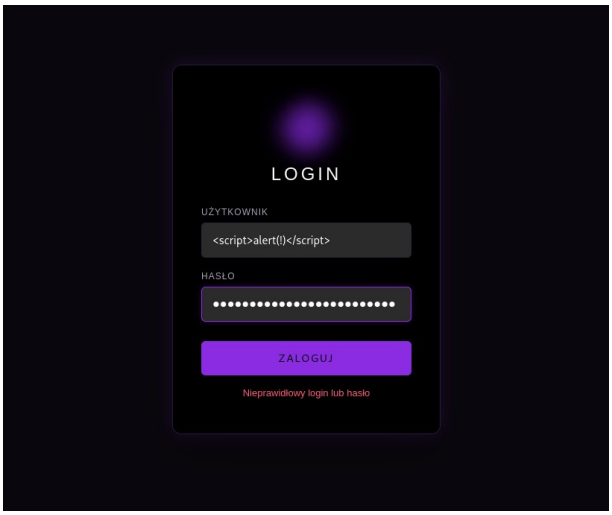


FIGURE 7 | XSS-shaped input

Interpretation

The page displays the submitted string and a generic login failure. No JavaScript alert or other script execution is visible in the captured evidence.

Conclusion: XSS was tested, but a vulnerability was not confirmed.

The login request was then captured in Burp Proxy so the request method, target path and form body could be inspected before configuring Intruder.

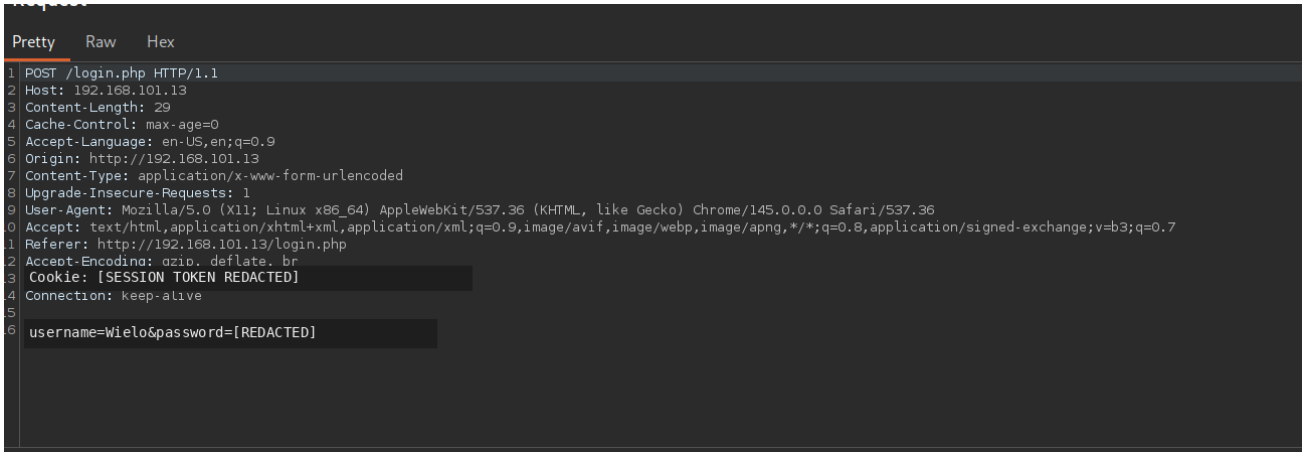


FIGURE 8 | Captured POST request (publication-safe copy)

The request targets POST /login.php and uses form-encoded credentials. The session token and password value are redacted in this report copy.

SCOPE NOTE

The input probe and credential-guessing run are separate tests. The XSS-shaped input does not, by itself, indicate an authentication bypass or a confirmed cross-site scripting vulnerability.

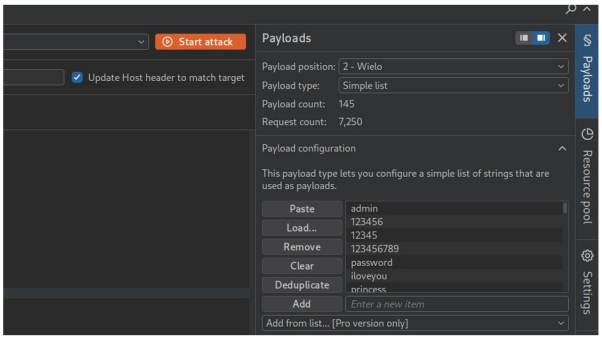
05 / BURP SUITE INTRUDER

Automated Credential-Guessing Test

Burp Suite Intruder was configured with the Cluster bomb attack type and two payload positions, allowing combinations from the supplied username and password lists to be tested against the lab endpoint.



Request positions (session token redacted)



Payload list and request count

FIGURE 9 | Intruder Cluster bomb configuration

Reviewing the result set

View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length
46	nobody	Kiwi	200	3			6513
47	Mixu	Kiwi	302	505			402
48	nobody4	Kiwi	200	3			6513
49	gnats	tinkerbell	200	4			6513
50	Wielo	tinkerbell	200	553			6513
51	landscape	tinkerbell	200	1			6513
52	libstoragemgmt	tinkerbell	200	2			6513
53	Socik	tinkerbell	200	454			6513
54	nobody	tinkerbell	200	2			6513
55	Mixu	tinkerbell	200	582			6513
56	nobody4	tinkerbell	200	3			6513
57	gnats	charlie	200	2			6513
58	Wielo	charlie	200	540			6513
59	landscape	charlie	200	8			6513
60	libstoragemgmt	charlie	200	2			6513
61	Socik	charlie	200	539			6513
62	nobody	charlie	200	367			6513
63	Mixu	charlie	200	544			6513
64	nobody4	charlie	200	2			6513
65	gnats	Samolocik	200	1			6513
66	Wielo	Samolocik	200	570			6513
67	landscape	Samolocik	200	2			6513
68	libstoragemgmt	Samolocik	200	3			6513
69	Socik	Samolocik	302	598			402

FIGURE 10 | Intruder response differences

The shorter response lengths strongly suggest potential authentication compromise. Confirm candidate requests against runes_auth.log before treating them as successful logins.

> **ANALYST DECISION**
>
> Use Intruder status/length differences to identify candidate responses. Confirm each candidate against the application's LOGIN_SUCCESS event, then correlate the event timestamp with access.log.

06 / LOG TRIAGE

Evidence Review: High-Volume Requests and Auth Outcomes

The captured access-log excerpt shows repeated POST /login.php requests from a single recorded source address over a short period. By itself, this indicates request volume, not the outcome of every login attempt.

```
192.168.101.13 - - [14/Oct/2026:11:38:09 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:10 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:11 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:11 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:11 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:12 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:12 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:12 -0500] "POST /login.php HTTP/1.1" 200 2237 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:13 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:14 -0500] "POST /login.php HTTP/1.1" 200 2237 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:14 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:15 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:15 -0500] "POST /login.php HTTP/1.1" 200 2237 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:16 -0500] "POST /login.php HTTP/1.1" 200 2237 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:16 -0500] "POST /login.php HTTP/1.1" 200 2237 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:17 -0500] "POST /login.php HTTP/1.1" 200 2237 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:19 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:20 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:22 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:22 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:22 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:25 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:25 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:26 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:26 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:28 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:29 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:29 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:30 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:31 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:31 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_6
192.168.101.13 - - [14/Oct/2026:11:38:32 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:32 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:34 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
192.168.101.13 - - [14/Oct/2026:11:38:35 -0500] "POST /login.php HTTP/1.1" 200 2238 "http://192.168.101.13/login.php" "Mozilla/5.0 (X11; Linux x86_
```

FIGURE 11 | Apache access.log during the test

Repeated POST requests occur at short intervals. Some responses return HTTP 200, while selected requests return HTTP 302. The 302 responses are candidate successful logins to validate with application telemetry.

The application-authentication excerpt gives the missing semantic detail: failures include a reason field, and successful checks are explicitly recorded as LOGIN_SUCCESS. The visible source address is 192.168.101.13.


```
{"timestamp":"2026-10-14T16:38:29+00:00","event":"LOGIN_FAILURE","username":"nobody4","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:30+00:00","event":"LOGIN_FAILURE","username":"gnats","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:32+00:00","event":"LOGIN_SUCCESS","username":"Wielo","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:38:32+00:00","event":"LOGIN_FAILURE","username":"nobody4","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:32+00:00","event":"LOGIN_FAILURE","username":"gdm","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:32+00:00","event":"LOGIN_FAILURE","username":"landscape","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:34+00:00","event":"LOGIN_FAILURE","username":"libstoragemgmt","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:35+00:00","event":"LOGIN_FAILURE","username":"Socik","ip":"192.168.101.13","reason":"invalid_password"}
{"timestamp":"2026-10-14T16:38:35+00:00","event":"LOGIN_FAILURE","username":"geoclue","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:36+00:00","event":"LOGIN_FAILURE","username":"nobody","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:37+00:00","event":"LOGIN_FAILURE","username":"ntp","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:38+00:00","event":"LOGIN_FAILURE","username":"Mixu","ip":"192.168.101.13","reason":"invalid_password"}
{"timestamp":"2026-10-14T16:38:39+00:00","event":"LOGIN_FAILURE","username":"nobody4","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:40+00:00","event":"LOGIN_FAILURE","username":"gnats","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:41+00:00","event":"LOGIN_FAILURE","username":"gnats","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:42+00:00","event":"LOGIN_FAILURE","username":"Wielo","ip":"192.168.101.13","reason":"invalid_password"}
{"timestamp":"2026-10-14T16:38:43+00:00","event":"LOGIN_FAILURE","username":"nuucp","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:44+00:00","event":"LOGIN_FAILURE","username":"landscape","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:45+00:00","event":"LOGIN_FAILURE","username":"libstoragemgmt","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:46+00:00","event":"LOGIN_SUCCESS","username":"Wielo","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:38:47+00:00","event":"LOGIN_FAILURE","username":"Socik","ip":"192.168.101.13","reason":"invalid_password"}
{"timestamp":"2026-10-14T16:38:47+00:00","event":"LOGIN_FAILURE","username":"nxautomation","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:48+00:00","event":"LOGIN_FAILURE","username":"nobody","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:49+00:00","event":"LOGIN_FAILURE","username":"gnome-initial-setup","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:50+00:00","event":"LOGIN_FAILURE","username":"Mixu","ip":"192.168.101.13","reason":"invalid_password"}
{"timestamp":"2026-10-14T16:38:51+00:00","event":"LOGIN_FAILURE","username":"gdm","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:52+00:00","event":"LOGIN_FAILURE","username":"nobody4","ip":"192.168.101.13","reason":"unknown_user"}
{"timestamp":"2026-10-14T16:38:52+00:00","event":"LOGIN_FAILURE","username":"gopher","ip":"192.168.101.13","reason":"unknown_user"}
```


FIGURE 12 | Authentication events during the test

The excerpt contains a dense sequence of LOGIN_FAILURE events and successful events interleaved with them. The mix of unknown_user and invalid_password reasons helps characterize the tested combinations; neither reason alone identifies intent.

> **SOURCE-IP CAVEAT**
>
> 192.168.101.13 is a private address recorded in this lab. It is an observed source indicator for this dataset, not evidence of external attribution or malicious reputation. Confirm network topology before using IP attribution in a real incident.

07 / TIMELINE

Correlation: Successful Authentication in the Investigation Window

The investigation window starts at 2026-10-14 16:32:15 UTC, the earliest failed-login timestamp selected for this analysis. This timestamp is used as the investigation anchor; the analysis focuses on events recorded from that point onward.


```
{"timestamp":"2026-10-14T16:38:32+00:00","event":"LOGIN_SUCCESS","username":"Wielo","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:38:46+00:00","event":"LOGIN_SUCCESS","username":"Wielo","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:39:20+00:00","event":"LOGIN_SUCCESS","username":"Mixu","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:40:13+00:00","event":"LOGIN_SUCCESS","username":"Socik","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T16:57:31+00:00","event":"LOGIN_SUCCESS","username":"Mixu","ip":"192.168.101.13"}
{"timestamp":"2026-10-14T17:18:36+00:00","event":"LOGIN_SUCCESS","username":"Socik","ip":"192.168.101.13"}
```


FIGURE 13 | Filtered LOGIN_SUCCESS events

Filtering from the selected UTC anchor shows six successful authentication events in the retained output. The visible account names are Wielo, Mixu and Socik.


```
┌──(kali㉿kali)-[~]
└─$ awk '$4 ~ /^\[14\/Oct\/2026:/ && $4 >= "[14/Oct/2026:11:32:15" && $6=="\"POST" && $7=="/login.php" && $9==302' /var/log/ap
192.168.101.13 - - [14/Oct/2026:11:38:31 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/
ri/537.36"
192.168.101.13 - - [14/Oct/2026:11:38:45 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/
ri/537.36"
192.168.101.13 - - [14/Oct/2026:11:39:20 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/
ri/537.36"
192.168.101.13 - - [14/Oct/2026:11:40:12 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/
ri/537.36"
192.168.101.13 - - [14/Oct/2026:11:57:31 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/
ri/537.36"
192.168.101.13 - - [14/Oct/2026:12:18:36 -0500] "POST /login.php HTTP/1.1" 302 402 "http://192.168.101.13/login.php" "Mozilla/
ri/537.36"
```


FIGURE 14 | Filtered HTTP 302 responses

The HTTP 302 entries correspond to the six application-level successes identified in the filtered output. This cross-source agreement supports that the redirects correspond to successful login handling in this application.

> CORRELATION RESULT
> Within the captured investigation window, six LOGIN_SUCCESS events are supported by corresponding POST /login.php responses with HTTP 302. The lab test therefore demonstrated successful authentication for three test usernames. This is evidence of accepted credentials in the lab, not evidence of post-login data access or real-world account compromise.

08 / FINDINGS

SOC Assessment and Detection Logic

Evidence-based assessment

Assessment	Conclusion	Evidence / confidence
Automated credential guessing	Consistent with the controlled Intruder run	Known Cluster bomb test configuration plus repeated POST requests and numerous auth failures. High confidence for the lab scenario.
Successful authentication during test	Observed	Six LOGIN_SUCCESS events are present in the filtered output, with corresponding HTTP 302 responses in Apache access.log. High confidence for the captured events.
Source identification	Partially established	192.168.101.13 is the recorded private IP. The logs alone do not identify a person or independent device.
Post-authentication impact	Not established	The evidence confirms authentication/redirect handling; it does not show sensitive-data access, persistence or other impact.
XSS vulnerability	Not confirmed	A script-like string was submitted, but no script execution is visible in the screenshots.

Starter detection logic for a lab

RULE A — LOGIN FAILURE BURST

Raise a candidate alert when one source IP generates at least 10 LOGIN_FAILURE events in a rolling 60-second window. Treat this as a lab threshold; tune it against normal activity before using it operationally.

RULE B — SUCCESS AFTER A BURST

Increase the alert priority when LOGIN_SUCCESS occurs from the same source IP or against a targeted account within five minutes of the failure-burst alert. Preserve the linked failures and success as evidence for review.

09 / ANALYST REFERENCE

Appendix: Reproducible Triage Commands

These read-only commands reproduce the core checks shown in the report. Run them against the retained lab logs. The investigation-anchor filter is specific to the captured test on 14 October 2026; update the timestamp for later runs.

1. Count authentication outcomes

```
grep -c '"event":"LOGIN_FAILURE"' /var/log/apache2/runes_auth.log
grep -c '"event":"LOGIN_SUCCESS"' /var/log/apache2/runes_auth.log
```

Purpose: establish the total number of recorded failures and successes in the selected file. These totals should be interpreted within the relevant investigation window.

2. Filter successful authentications from the UTC anchor

```
awk -F'"' '$2 >= "2026-10-14T16:32:15+00:00" && $4 == "LOGIN_SUCCESS" { print }'
/var/log/apache2/runes_auth.log
```

Purpose: select application-level success events whose timestamp is at or after the chosen investigation anchor. In this JSON format, the timestamp is field $2 and the event value is field $4 when splitting on double quotes.

3. Count login submissions in Apache access.log

```
grep '"POST /login.php HTTP/1.1"' /var/log/apache2/access.log | wc -l
```

Purpose: quantify observed login requests. For a proper per-source metric, filter by the client IP and by the selected time window as well.

FINAL ANALYST NOTE

A useful SOC conclusion is built from the chain of evidence: high-volume login requests → authentication failures → successful authentication events → matching HTTP redirects → assessment of activity after login. Keep observed facts separate from hypotheses and explicitly document gaps in telemetry.